Programação Aplicada em .Net

Aula 3

Prof. Me. Lucas Teodoro dos Santos

UNIP

31 de Agosto de 2026

Roteiro da Aula

1. DTO
2. Imutabilidade
3. Records

UNIP
UNIVERSIDADE PAULISTA

Problema

Em uma aplicação, os objetos costumam possuir várias propriedades, pois representam todas as informações que o sistema precisa conhecer.

Entretanto, uma operação específica nem sempre precisa utilizar todos esses dados.

Por exemplo, para lançar uma nota, precisamos saber:

- Qual aluno receberá a nota;
- Qual é o valor da nota.

Não precisamos receber nome, curso, matrícula, senha ou as demais notas do aluno.

UNIP
UNIVERSIDADE PAULISTA

Entidade Aluno

Considere uma classe que representa um aluno dentro do sistema:

```
1           public class Aluno
2           {
3               public int Id { get; set; }
4               public string Nome { get; set; }
5               public string Matricula { get; set; }
6               public string Curso { get; set; }
7               public string SenhaPortal { get; set; }
8               public List<double> Notas { get; set; }
9           }
```

Essa classe possui todas as informações que o sistema pode precisar sobre um aluno.

UNIP
UNIVERSIDADE PAULISTA

Utilização do objeto completo

Podemos utilizar a classe `Aluno` em diversas partes da aplicação. Porém, em algumas situações, enviar ou receber o objeto completo não é uma boa escolha.

Isso pode causar alguns problemas:

- Um payload maior, com dados que não serão utilizados;
- Maior quantidade de dados trafegando entre as partes da aplicação;
- Exposição de informações internas, como uma senha;
- Possibilidade de alteração de campos indevidos;
- Falta de clareza sobre quais dados uma operação realmente precisa.

UNIP
UNIVERSIDADE PAULISTA

DTO - *Data Transfer Object*

DTO significa *Data Transfer Object*, ou **Objeto de Transferência de Dados**.

Um DTO é uma classe criada para transportar os dados necessários entre duas partes da aplicação.

Em vez de utilizar sempre a entidade completa, criamos uma classe específica para cada necessidade de comunicação.

UNIP
UNIVERSIDADE PAULISTA

Entidade x DTO

Entidade	**DTO**
Representa os dados completos do sistema.	Representa os dados necessários para uma situação específica.
Pode estar relacionada ao banco de dados.	Não precisa representar uma tabela do banco.
Pode conter dados internos e relacionamentos.	Contém apenas os dados que serão recebidos ou enviados.
Exemplo: `Aluno`.	Exemplos: `LancarNotaDto` e `BoletimAlunoDto`.

Um DTO para cada necessidade

Uma mesma entidade pode possuir vários DTOs.

No caso do aluno, podemos precisar de formatos diferentes de dados:

- Um DTO para cadastrar um aluno;
- Um DTO para alterar seus dados;
- Um DTO para lançar uma nota;
- Um DTO para exibir o boletim;
- Um DTO para mostrar apenas um resumo do aluno.

Cada DTO deve possuir somente as propriedades necessárias para sua finalidade.

UNIP
UNIVERSIDADE PAULISTA

Uma operação com uma necessidade específica

Considere a operação de lançar uma nota para um aluno.

Para realizar essa ação, o sistema precisa saber apenas:

- Qual aluno receberá a nota;
- Qual é o valor da nota.

Entretanto, já existe no sistema a classe `Aluno`, que possui essas informações e diversas outras propriedades.

Seria possível reutilizar essa classe para representar o lançamento da nota?

UNIP
UNIVERSIDADE PAULISTA

Reutilizando a classe `Aluno`

Para lançar uma nota, poderíamos criar um objeto do tipo `Aluno` preenchendo apenas algumas propriedades:

```
var aluno = new Aluno
{
    Id = 1,
    Notas = new List<double> { 8.5 }
};
```

Tecnicamente, isso funciona: o sistema consegue identificar o aluno e acessar a nota informada.

Mas esse objeto não representa um aluno completo. Ele está sendo utilizado para representar outra intenção: lançar uma nota.

Problemas ao reutilizar a entidade

A classe `Aluno` foi criada para representar os dados de um aluno, e não uma operação de lançamento de nota.

Ao utilizá-la dessa forma, surgem algumas dúvidas:

- Por que um aluno está sendo enviado sem nome, curso e matrícula?
- A lista `Notas` representa uma nova nota ou todas as notas do aluno?
- A intenção é adicionar, alterar ou consultar notas?

A entidade não comunica claramente o que a operação precisa fazer.

UNIP
UNIVERSIDADE PAULISTA

E se outros campos forem informados?

Como a operação recebe um objeto do tipo `Aluno`, também seria possível informar campos que não fazem parte do lançamento de nota.

Por exemplo:

- Nome;
- Matrícula;
- Curso;
- Senha do portal;
- Lista completa de notas.

Precisamos de uma classe que aceite somente os dados necessários para essa operação.

UNIP
UNIVERSIDADE PAULISTA

DTO de entrada

Um **DTO de entrada** representa os dados que uma operação precisa receber.

Ele define de forma clara quais informações podem ser informadas por quem utiliza a aplicação.

No lançamento de uma nota, o DTO deve receber somente o identificador do aluno e o valor da nota.

Dessa forma, a comunicação fica mais clara e campos que não fazem parte da operação não precisam ser recebidos.

UNIP
UNIVERSIDADE PAULISTA

Exemplo: DTO de entrada

```
1           public class LancarNotaDto
2           {
3               public int AlunoId { get; set; }
4               public double Nota { get; set; }
5           }
```

Esse DTO representa exatamente os dados necessários para a operação:

- `AlunoId`: identifica o aluno que receberá a nota;
- `Nota`: informa o valor que será lançado.

Agora, a classe utilizada comunica claramente a intenção da operação.

UNIP
UNIVERSIDADE PAULISTA

E quando os dados precisam ser devolvidos?

O DTO não é utilizado apenas para controlar os dados que uma operação recebe.

Também precisamos decidir quais dados devem ser devolvidos ao final de uma operação.

No caso do boletim de um aluno, queremos apresentar informações como:

- Nome do aluno;
- Curso;
- Média das notas;
- Situação final.

Não é necessário devolver todos os dados existentes na entidade `Aluno`.

DTO de saída

Um **DTO de saída** representa os dados que uma operação deve devolver.

Ele permite definir exatamente quais informações serão apresentadas como resultado de uma operação.

No boletim, por exemplo, não precisamos devolver a senha do portal, a matrícula ou a lista completa de notas.

O DTO de saída contém apenas os dados que fazem sentido para quem irá visualizar o resultado.

UNIP
UNIVERSIDADE PAULISTA

Exemplo: DTO de saída

```
public class BoletimAlunoDto
{
    public string NomeAluno { get; set; }
    public string Curso { get; set; }
    public double Media { get; set; }
    public string Situacao { get; set; }
}
```

O sistema utiliza os dados da entidade `Aluno` para montar o DTO que será devolvido:

```
var boletim = new BoletimAlunoDto
{
    NomeAluno = aluno.Nome,
    Curso = aluno.Curso,
    Media = aluno.Notas.Average(),
    Situacao = "Aprovado"
};
```

O DTO de saída pode incluir dados calculados, como a média e a situação.

Ferramentas de mapeamento

Em exemplos pequenos, podemos criar o DTO manualmente:

```
1           var boletim = new BoletimAlunoDto
2           {
3               NomeAluno = aluno.Nome,
4               Curso = aluno.Curso,
5               Media = aluno.Notas.Average(),
6               Situacao = "Aprovado"
7           };
```

Em aplicações maiores, com muitas entidades e DTOs, esse mapeamento pode se repetir em diversos pontos do sistema.

Existem ferramentas de mapeamento de objetos que podem automatizar parte da conversão entre entidades e DTOs, principalmente quando as propriedades possuem nomes e tipos correspondentes.

Essas ferramentas não substituem os DTOs: elas apenas ajudam a realizar a conversão entre os objetos.

Exemplos de ferramentas de mapeamento

Existem bibliotecas que auxiliam no mapeamento entre entidades, DTOs e outros objetos da aplicação.

Alguns exemplos são:

- **AutoMapper**: uma das bibliotecas de mapeamento mais conhecidas no ecossistema .NET;
- **Mapster**: biblioteca que permite realizar mapeamentos de objetos com uma sintaxe simples;
- **Mapperly**: ferramenta que gera código de mapeamento durante a compilação.

UNIP
UNIVERSIDADE PAULISTA

Entrada x saída

DTO de entrada	**DTO de saída**
Representa os dados que uma operação recebe.	Representa os dados que uma operação devolve.
Controla o que pode ser informado ou alterado.	Controla o que pode ser exibido.
Exemplo: `LancarNotaDto`.	Exemplo: `BoletimAlunoDto`.
Possui somente os campos necessários para executar a ação.	Possui somente os campos necessários para apresentar o resultado.

Quando utilizar DTO?

- Quando não queremos expor todos os dados de uma entidade;
- Quando uma operação recebe apenas alguns campos;
- Quando queremos controlar quais dados podem ser alterados;
- Quando precisamos apresentar os dados em um formato específico;
- Quando precisamos incluir dados calculados em uma resposta;
- Quando diferentes operações precisam de diferentes conjuntos de dados.

UNIP
UNIVERSIDADE PAULISTA

O que é imutabilidade?

Um objeto é **imutável** quando o seu estado não pode ser alterado depois que ele é criado.

Isso significa que suas propriedades recebem valores no momento da criação do objeto e permanecem iguais durante toda a sua existência.

Por exemplo, um objeto que representa o resumo de um aluno pode ser criado com nome, curso e média. Depois disso, esses dados não devem ser modificados.

UNIP
UNIVERSIDADE PAULISTA

Objetos mutáveis

Em uma classe com propriedades que possuem `set`, os valores podem ser alterados a qualquer momento:

```
public class AlunoResumo
{
    public string Nome { get; set; }
    public string Curso { get; set; }
    public double Media { get; set; }
}
```

```
var aluno = new AlunoResumo
{
    Nome = "Yasmin",
    Curso = "ADS",
    Media = 8.5
};

aluno.Media = 5.0;
```

Nesse caso, o objeto é **mutável**, pois sua média foi alterada após sua criação.

Por que utilizar imutabilidade?

Quando um objeto pode ser alterado em qualquer parte do código, pode ser mais difícil entender em que momento seus dados mudaram.

A imutabilidade ajuda a reduzir:

- Alterações inesperadas nos dados;
- Erros causados pelo compartilhamento de objetos;
- Dificuldade para identificar onde um valor foi alterado;
- Código difícil de prever e manter.

Um objeto imutável mantém os mesmos dados durante toda a sua existência.

UNIP
UNIVERSIDADE PAULISTA

Opções para controlar alterações

Em classes, existem diferentes formas de controlar como uma propriedade pode receber ou alterar valores.

- Sem `set`: o valor é definido, normalmente, pelo construtor e não pode mais ser alterado;
- `private set`: o valor pode ser alterado apenas dentro da própria classe;
- `init`: o valor pode ser definido na criação do objeto, mas não pode ser alterado posteriormente.

Cada opção atende a uma necessidade diferente. Para imutabilidade, as opções mais utilizadas são propriedades sem `set` ou com `init`.

UNIP
UNIVERSIDADE PAULISTA

Classe imutável sem `set`

Sem `set`, as propriedades podem ser definidas pelo construtor, mas não podem ser alteradas depois:

```
1           public class AlunoResumoClasse
2           {
3               public string Nome { get; }
4               public string Curso { get; }
5               public double Media { get; }
6
7               public AlunoResumoClasse(string nome, string curso, double media)
8               {
9                   Nome = nome;
10                  Curso = curso;
11                  Media = media;
12              }
13          }
```

Essa é uma forma de criar uma classe imutável.

Propriedades com `init`

A palavra `init` permite atribuir valores durante a criação do objeto, inclusive utilizando inicializadores de objeto:

```
1           public class AlunoResumoClasse
2           {
3               public string Nome { get; init; }
4               public string Curso { get; init; }
5               public double Media { get; init; }
6           }
```

```
1           var aluno = new AlunoResumoClasse
2           {
3               Nome = "Matheus",
4               Curso = "ADS",
5               Media = 5.5
6           };
```

Após a criação, essas propriedades não podem mais ser alteradas.

Propriedades com `private set`

Com `private set`, outras classes não podem alterar a propriedade diretamente:

```
1           public class Aluno
2           {
3               public string Nome { get; private set; }
4               public double Media { get; private set; }
5
6               public void AtualizarMedia(double novaMedia)
7               {
8                   Media = novaMedia;
9               }
10          }
```

```
1           aluno.Media = 8.5; // Nao permitido
2
3           aluno.AtualizarMedia(8.5); // Permitido
```

A classe continua podendo alterar seus próprios dados. Portanto, `private set` representa encapsulamento e controle de alteração, mas não imutabilidade completa.

UNIP UNIVERSIDADE PAULISTA

Comparando as opções

Opção	**Quando pode receber valor?**	**Pode ser alterada depois?**
Sem `set`	Normalmente pelo construtor.	Não.
`init`	No construtor ou na criação com inicializador de objeto.	Não.
`private set`	Na criação e dentro da própria classe.	Sim, pela própria classe.

Records

Records são tipos do C# adequados para representar objetos cujo objetivo principal é armazenar dados.

Assim como as classes, records podem possuir propriedades, métodos e construtores. Porém, eles possuem recursos que facilitam a criação de objetos orientados a dados.

Records também são uma ferramenta que auxilia na aplicação da **imutabilidade**, pois permitem criar objetos cujos valores são definidos na criação e não devem ser alterados posteriormente.

Além disso, oferecem recursos como comparação por valor, desconstrução e criação de novas versões com `with`.

Por esse motivo, são frequentemente utilizados para DTOs, respostas, configurações e resultados de consultas.

Formas de criar um Record

Um record pode ser declarado de duas formas principais.

1. Forma posicional: mais curta, indicada quando o objeto possui somente propriedades de dados.

```
1           public record AlunoResumoRecord(string Nome, string Curso, double Media);
```

2. Forma com corpo: permite declarar as propriedades explicitamente e adicionar construtores ou métodos.

```
1           public record AlunoResumoRecord
2           {
3               public string Nome { get; init; }
4               public string Curso { get; init; }
5               public double Media { get; init; }
6           }
```

Criando objetos a partir dos Records

Na forma posicional, os valores são informados no construtor:

```
var aluno = new AlunoResumoRecord("Renato", "ADS", 7.5);
```

Na forma com corpo, podemos utilizar um inicializador de objeto:

```
var aluno = new AlunoResumoRecord
{
    Nome = "Renato",
    Curso = "ADS",
    Media = 7.5
};
```

Nas duas formas, o uso de `init` ou do record posicional impede a alteração das propriedades após a criação do objeto.

Desconstrução de Records

Records posicionais permitem extrair seus valores para variáveis de forma simples. Esse recurso é chamado de **desconstrução**.

```
1           var aluno = new AlunoResumo("Julia", "ADS", 8.5);
2
3           var (nome, curso, media) = aluno;
```

Após a desconstrução, cada propriedade do record foi atribuída a uma variável:

```
1           Console.WriteLine(nome);
2           Console.WriteLine(curso);
3           Console.WriteLine(media);
```

A ordem das variáveis segue a ordem das propriedades declaradas no record posicional.

Criando uma nova versão do objeto com `with`

Records imutáveis não permitem alterar as propriedades do objeto original depois de sua criação.

Quando precisamos representar uma alteração, utilizamos `with` para criar uma nova instância:

```
var aluno = new AlunoResumoRecord("Ana", "ADS", 8.5);

var alunoAtualizado = aluno with
{
    Media = 9.0
};
```

O objeto `aluno` não foi alterado e continua com média `8,5`.

O objeto `alunoAtualizado` é uma nova instância que possui os mesmos dados, exceto pela média atualizada para `9,0`.

Comparação entre Classes e Records

Classe	**Record**
Pode representar dados e comportamentos do sistema.	É especialmente adequado para representar dados.
A comparação padrão verifica se são a mesma instância.	A comparação padrão considera os valores das propriedades.
Pode ser mutável ou imutável.	Pode ser mutável ou imutável, mas é frequentemente usado de forma imutável.
Exige mais código para construtor e propriedades somente leitura.	Oferece uma sintaxe mais curta para objetos de dados.

Comparação por valor: Classe

Em uma classe comum, dois objetos com os mesmos dados continuam sendo instâncias diferentes.

```
1            var aluno1 = new AlunoResumoClasse("Jose", "ADS", 8.5);
2
3            var aluno2 = new AlunoResumoClasse("Jose", "ADS", 8.5);
4
5            Console.WriteLine(aluno1 == aluno2);
```

Resultado:

```
False
```

A comparação padrão de classes verifica se as variáveis apontam para a mesma instância na memória.

Comparação por valor: Record

Em um record, a comparação padrão considera os valores de suas propriedades.

```
1            var aluno1 = new AlunoResumoRecord("Jose", "ADS", 8.5);
2
3            var aluno2 = new AlunoResumoRecord("Jose", "ADS", 8.5);
4
5            Console.WriteLine(aluno1 == aluno2);
```

Resultado:

```
True
```

Apesar de serem objetos diferentes, os records possuem os mesmos valores e, por isso, são considerados iguais.

Imutabilidade dos Records

Um `record` não é obrigatoriamente imutável. Podemos criar propriedades com `set`, permitindo alterações após a criação do objeto:

```
1           public record AlunoResumoRecord
2           {
3               public string Nome { get; set; }
4               public double Media { get; set; }
5           }
```

Porém, os recursos fornecidos por records favorecem naturalmente o uso de objetos imutáveis.

Por exemplo, na forma posicional, as propriedades são criadas com `init`, recebendo valores durante a criação do objeto:

```
1           public record AlunoResumoRecord(string Nome, double Media);
```

Nesse formato, os valores não podem ser alterados posteriormente. Quando uma mudança é necessária, criamos uma nova instância com `with`.

Records e DTOs

DTOs representam dados que entram ou saem de uma operação.

Principalmente nos DTOs de saída, esses dados normalmente não precisam ser alterados depois que são criados.

Nesse cenário, records são uma opção adequada porque:

- Representam dados de forma concisa;
- Favorecem a imutabilidade;
- Comparam objetos pelos valores;
- **Deixam clara a intenção de transportar dados.**

UNIP
UNIVERSIDADE PAULISTA

DTO de saída utilizando record

Podemos representar o boletim do aluno com um record:

```
1           public record BoletimAlunoDto(string NomeAluno,string Curso,double Media, string Situacao);
```

```
1           var boletim = new BoletimAlunoDto("Ana Souza","ADS",8.5,"Aprovado");
```

Esse DTO representa um resultado pronto para ser apresentado e não precisa ser alterado depois de criado.

Quando utilizar classes e quando utilizar records?

Classe	**Record**
Quando o objeto possui comportamento e pode mudar ao longo do tempo. Exemplo: entidade `Aluno`, cujas notas podem ser lançadas ou alteradas. Útil para entidades e objetos com regras de negócio.	Quando o objeto representa principalmente um conjunto de dados. Exemplo: `BoletimAlunoDto`, que representa um resultado pronto. Útil para DTOs, respostas de consultas, objetos de configuração e outros dados que não devem ser alterados após sua criação.

Resumo Geral da Aula

- DTOs representam e transportam somente os dados necessários para uma operação específica;
- DTOs de entrada controlam o que uma operação recebe;
- DTOs de saída controlam o que uma operação devolve;
- O mapeamento transforma entidades em DTOs;
- Imutabilidade impede que um objeto seja alterado após sua criação;
- Classes podem controlar alterações por meio de propriedades sem `set`, com `init` ou com `private set`;
- Records são adequados para objetos focados em dados e possuem recursos que favorecem a imutabilidade;
- Records realizam comparação por valor e permitem criar novas versões de objetos com `with`;
- DTOs podem ser representados por records.

Obrigado!

Dúvidas?